Embedded Software Project: STM32 Peripheral Drivers

Comprehensive Technical Analysis & Implementation Report (GPIO, USART, I2C, SPI)

Author: Muhammed Saeed

Target Hardware: STM32F446RE / STM32F401RE (NUCLEO-F446RE, NUCLEO-F401RE, STM32F4-Discovery)

Core Architecture: ARM® Cortex®-M4 with FPU @ 180 MHz (STM32F446RE) / 84 MHz (STM32F401RE)

Reference Manuals: ST RM0390 (STM32F446xx), ST RM0368 (STM32F401xx), ARM Cortex-M4 PM0214

Base Repository: https://github.com/ghostcoder911/STM32F446RE_Drivers.git

---

Executive Summary

Bare-metal firmware development forms the foundation of embedded software engineering. By bypassing high-level vendor libraries (such as STM32Cube HAL or Arduino abstractions), developers gain deterministic control over hardware registers, timing constraints, interrupt latencies, and silicon memory footprints.

This technical report delivers an in-depth analysis of a complete bare-metal peripheral driver suite developed from scratch for the STMicroelectronics STM32F446RE/STM32F401RE microcontrollers. Four core communication and control peripherals are systematically examined:

1. **GPIO (General Purpose Input/Output)**
2. **USART (Universal Synchronous/Asynchronous Receiver Transmitter)**
3. **I2C (Inter-Integrated Circuit)**
4. **SPI (Serial Peripheral Interface)**

Along with peripheral analysis, this document details the board adaptation layer (`BoardConfig.h`) for the **NUCLEO-F446RE / NUCLEO-F401RE** platform, resolves critical silicon bugs identified during driver inspection (including NVIC register indexing and 16-bit SPI pointer arithmetic), presents six mandatory application examples, and shares reflective takeaways on embedded driver design.

---

1. STM32 MCU Architecture & Bus Topology

The STM32F446RE is built on a 32-bit multi-AHB bus matrix interconnecting the ARM Cortex-M4 CPU with on-chip SRAM, Flash, and peripheral buses:

- **Flash Memory:** 512 KB starting at `0x08000000U`
- **SRAM:** 128 KB total (SRAM1: 112 KB at `0x20000000U`, SRAM2: 16 KB at `0x2001C000U`)
- **System ROM (Bootloader):** Starting at `0x1FFF0000U`
- **AHB1 Bus (up to 180 MHz):** Hosts high-speed system controllers and GPIO Ports A through H (`0x40020000U` to `0x40021C00U`), RCC (`0x40023800U`), DMA1/2.
- **APB1 Bus (up to 45 MHz):** Low-speed peripheral bus hosting TIM2-TIM7, SPI2/3, USART2/3, UART4/5, I2C1-I2C3.
- **APB2 Bus (up to 90 MHz):** High-speed peripheral bus hosting TIM1/TIM8, USART1/6, SPI1/4, SYSCFG, EXTI, ADC1-ADC3.

In default bare-metal reset condition, the microcontroller runs from the internal High-Speed Internal (HSI) RC oscillator at **16 MHz**. Both APB1 and APB2 buses operate at 16 MHz (prescalers = 1). Dynamic bus clock determination is orchestrated by the `stm32f446xx_rcc_driver`, ensuring baud rate generators and I2C prescalers dynamically match system frequency.

---

2. GPIO Driver Deep Dive

2.1 Hardware Register Architecture

The General Purpose I/O peripheral on the STM32F446RE controls up to 16 pins per port. Each port is represented in the driver by the C structure `GPIO_RegDef_t`:

```
typedef struct {
    __vo uint32_t MODER;    /*!< GPIO port mode register,               Address offset: 0x00 */
    __vo uint32_t OTYPER;   /*!< GPIO port output type register,        Address offset: 0x04 */
    __vo uint32_t OSPEEDR;  /*!< GPIO port output speed register,       Address offset: 0x08 */
    __vo uint32_t PUPDR;    /*!< GPIO port pull-up/pull-down register,  Address offset: 0x0C */
    __vo uint32_t IDR;      /*!< GPIO port input data register,         Address offset: 0x10 */
    __vo uint32_t ODR;      /*!< GPIO port output data register,        Address offset: 0x14 */
    __vo uint32_t BSRR;     /*!< GPIO port bit set/reset register,      Address offset: 0x18 */
    __vo uint32_t LCKR;     /*!< GPIO port configuration lock register, Address offset: 0x1C */
```

```
    __vo uint32_t AFR[2];   /*!< AFR[0]: AFRL (pins 0-7), AFR[1]: AFRH (pins 8-15), Offset: 0x20 */

} GPIO_RegDef_t;
```

2.2 Driver Data Structures & Hardware Abstraction

The driver encapsulates pin configuration into `GPIO_PinConfig_t` and binds it to a physical peripheral port through `GPIO_Handle_t`:

```
typedef struct {

    uint8_t GPIO_PinNumber;        /* 0 to 15 */

    uint8_t GPIO_PinMode;          /* 0: IN, 1: OUT, 2: ALTFN, 3: ANALOG, 4: IT_FT, 5: IT_RT, 6: IT_RFT */

    uint8_t GPIO_PinSpeed;         /* 0: LOW, 1: MEDIUM, 2: FAST, 3: HIGH */

    uint8_t GPIO_PinPuPdControl;   /* 0: NO_PUPD, 1: PU, 2: PD */

    uint8_t GPIO_PinOPType;        /* 0: Push-Pull, 1: Open-Drain */

    uint8_t GPIO_PinAltFunMode;    /* AF0 to AF15 */

} GPIO_PinConfig_t;


typedef struct {

    GPIO_RegDef_t *pGPIOx;         /* Base address: GPIOA, GPIOB, GPIOC, etc. */

    GPIO_PinConfig_t GPIO_PinConfig;

} GPIO_Handle_t;
```

2.3 Step-by-Step Initialization Sequence (`GPIO_Init`)

1. **Clock Activation:** The driver automatically invokes `GPIO_PeriClockControl(pGPIOHandle->pGPIOx, ENABLE)`, which sets the corresponding bit in `RCC->AHB1ENR` (e.g., bit 0 for GPIOA, bit 2 for GPIOC).

2. **Mode Configuration (`MODER`):**

- For non-interrupt modes (`Mode <= 3`), the 2-bit field corresponding to the pin is cleared and configured:

$$\text{MODER} \leftarrow (\text{MODER} \ \& \ \sim(0\text{x}3 \ll (2 \times \text{Pin}))) \ | \ (\text{Mode} \ll (2 \times \text{Pin}))$$

- For interrupt modes (`Mode > 3`):

- The pin mode is kept as Input (`00b`).

- **Edge Triggering:** Configures `EXTI->FTSR` (Falling Trigger) and/or `EXTI->RTSR` (Rising Trigger).

- **SYSCFG Routing:** Enables `SYSCFG` clock (`RCC->APB2ENR |= (1 << 14)`). It maps the port code (`0` for A, `1` for B, `2` for C) into `SYSCFG->EXTICR[Pin / 4]` at position `(Pin % 4) * 4`.

- **Interrupt Unmasking:** Enables event delivery by setting the corresponding bit in `EXTI->IMR`.

3. **Speed Configuration (`OSPEEDR`):** Programs low, medium, fast, or high slew rates into the 2-bit field `(2 * Pin)`.

4. **Resistor Configuration (`PUPDR`):** Programs floating (no pull), internal weak pull-up (~40 kΩ), or weak pull-down.

5. **Output Driver Configuration (`OTYPER`):** Programs single-ended push-pull (`0`) or open-drain (`1`).

6. **Alternate Function Multiplexing (`AFR`):** If `GPIO_MODE_ALTFN` is chosen, the driver routes the pin to an internal peripheral multiplexer:

- Pins 0–7 target `AFR[0]` (AFRL): shifted by `(Pin % 8) * 4`.

- Pins 8–15 target `AFR[1]` (AFRH): shifted by `((Pin - 8) % 8) * 4`.

2.4 I/O Data Transfer APIs

- **Atomic Pin Read (`GPIO_ReadFromInputPin`):** Reads the 32-bit `IDR` register, shifts right by `PinNumber`, and isolates bit 0:

$$\text{Value} = (\text{pGPIOx}\rightarrow\text{IDR} \gg \text{PinNumber}) \ \& \ 0\text{x}01$$

- **Atomic Pin Write (`GPIO_WriteToOutputPin`):** Modifies bit `PinNumber` in `ODR` via bitwise OR (set) or bitwise AND with complement (reset).
- **Pin Toggle (`GPIO_ToggleOutputPin`):** Evaluates `pGPIOx->ODR ^= (1 << PinNumber)` in a single cycle.

2.5 Interrupt & NVIC Management

The driver provides direct integration with the Cortex-M4 Nested Vectored Interrupt Controller (NVIC):

- **Enable/Disable (`GPIO_IRQInterruptConfig`):** Targets NVIC Interrupt Set-Enable (`NVIC_ISER0-2`) and Clear-Enable (`NVIC_ICER0-2`) registers.
- **Priority Programming (`GPIO_IRQPriorityConfig`):** Identifies the 8-bit priority slot in `NVIC_IPR[IRQNumber / 4]`. Because the STM32F446RE implements only the 4 most significant bits of each 8-bit priority register, the priority is shifted left by $(8 - 4) = 4$ bits plus the byte offset within the word.
- **ISR Clearing (`GPIO_IRQHandling`):** Clears the pending condition in `EXTI->PR` by writing a `1` (rc_w1 behavior).

---

3. USART Driver Deep Dive

3.1 Hardware Register Architecture

The Universal Synchronous/Asynchronous Receiver Transmitter (USART) provides full-duplex NRZ serial communications. It consists of:

- `SR` (Status Register): Flags `TXE` (bit 7), `TC` (bit 6), `RXNE` (bit 5), `IDLE` (bit 4), `ORE` (bit 3), `PE` (bit 0).
- `DR` (Data Register): 9-bit transmit/receive buffer.
- `BRR` (Baud Rate Register): 12-bit mantissa (`BRR[15:4]`) and 4-bit fraction (`BRR[3:0]`).
- `CR1`, `CR2`, `CR3`: Control registers for oversampling, word length, parity, stop bits, and interrupts.

3.2 Dynamic Baud Rate Generator (`USART_SetBaudRate`)

A critical engineering challenge in bare-metal USART drivers is accurate fractional baud rate computation without floating-point math overhead. The driver dynamically queries the APB bus clock:

- USART1 and USART6 reside on APB2 (`RCC_GetPCLK2Value()`).
- USART2, USART3, UART4, UART5 reside on APB1 (`RCC_GetPCLK1Value()`).

The standard formula for the clock divider is:

$$\text{USARTDIV} = \frac{f_{\text{PCLK}}}{8 \times (2 - \text{OVER8}) \times \text{BaudRate}}$$

To eliminate floating-point arithmetic while preserving fractional precision, the driver scales calculations by a factor of 100:

```
if (pUSARTx->CR1 & (1 << USART_CR1_OVER8)) {
    usartdiv = ((25 * PCLKx) / (2 * BaudRate)); /* OVER8 = 1 */
} else {
    usartdiv = ((25 * PCLKx) / (4 * BaudRate)); /* OVER16 = 0 */
}

M_part = usartdiv / 100;
F_part = usartdiv - (M_part * 100);

if (pUSARTx->CR1 & (1 << USART_CR1_OVER8)) {
    F_part = (((F_part * 8) + 50) / 100) & 0x07;  /* 3-bit fraction with rounding */
} else {
```

```
    F_part = (((F_part * 16) + 50) / 100) & 0x0F; /* 4-bit fraction with rounding */
}

pUSARTx->BRR = (M_part << 4) | F_part;
```

For 115200 baud at 16 MHz HSI with `OVER16 = 0`:

- $\text{USARTDIV} = 16{,}000{,}000 / (16 \times 115200) = 8.6805$
- `M_part` = 8 (`0x08`)
- `F_part` = $((68 \times 16) + 50) / 100 = 11$ (`0x0B`)
- Result written to `BRR` = `0x008B`, yielding an effective baud rate of $115{,}107$ bps (error $< 0.08\%$, well within RS-232 tolerances).

3.3 Data Transmission Mechanism

- **Polling Mode (`USART_SendData`):**

1. For each byte in buffer, spin-wait until `TXE` (Transmit Data Register Empty) is asserted:

`c

while (!USART_GetFlagStatus(pUSARTHandle->pUSARTx, USART_FLAG_TXE));

`

2. For 8-bit mode, load `pUSARTHandle->pUSARTx->DR = (*pTxBuffer & 0xFF)`.

3. Advance `pTxBuffer++` and decrement `Len`.

4. After all bytes are loaded, spin-wait for `TC` (Transmission Complete):

`c

while (!USART_GetFlagStatus(pUSARTHandle->pUSARTx, USART_FLAG_TC));

`

This ensures the shift register has fully transmitted the stop bit before the peripheral or pin multiplexer is altered.

3.4 Data Reception Mechanism

- **Polling Mode (`USART_ReceiveData`):**

1. Spin-wait until `RXNE` (Read Data Register Not Empty) is asserted.

2. If parity is disabled, read `*pRxBuffer = (uint8_t)(pUSARTx->DR & 0xFF)`.

3. If parity is enabled in 8-bit frame mode, bit 7 contains the hardware parity bit; the driver automatically masks the 7-bit user payload: `*pRxBuffer = (uint8_t)(pUSARTx->DR & 0x7F)`.

3.5 Interrupt-Driven State Machine

- `USART_SendDataIT` / `USART_ReceiveDataIT` store the application buffer pointer and length in `USART_Handle_t`, mark the state as `USART_BUSY_IN_TX` / `USART_BUSY_IN_RX`, and enable `TXEIE` / `RXNEIE`.
- `USART_IRQHandling` handles interrupts non-blockingly, servicing bytes until `Len == 0`, disabling interrupt masks, and notifying the application via `USART_ApplicationEventCallback`.

---

4. I2C Driver Deep Dive

4.1 Hardware Register Architecture & Protocol Timing

I2C is an open-drain, synchronous, bidirectional two-wire bus. In STM32 hardware, the I2C peripheral implements hardware clock stretching and strict protocol timing:

- `CR1`: Peripheral enable (`PE`), Start condition (`START`), Stop condition (`STOP`), Acknowledge enable (`ACK`).
- `CR2`: Peripheral clock frequency (`FREQ[5:0]`), Event/Buffer/Error interrupt enables (`ITEVTEN`, `ITBUFEN`, `ITERREN`).
- `SR1`: Start bit sent (`SB`), Address sent/matched (`ADDR`), Byte transfer finished (`BTF`), Stop received (`STOPF`), `RXNE`, `TXE`, Bus error (`BERR`), Arbitration lost (`ARLO`), Acknowledge failure (`AF`).
- `SR2`: Master/Slave (`MSL`), Bus busy (`BUSY`), Transmitter/Receiver (`TRA`).
- `CCR`: Clock control register for SCL generation.
- `TRISE`: Maximum rise time register.

4.2 Clock & Timing Calculations (`I2C_Init`)

1. **Peripheral Frequency (`CR2`):**

$$\text{CR2}\rightarrow\text{FREQ} = \frac{f_{\text{PCLK1}}}{1{,}000{,}000}$$

For a 16 MHz APB1 clock, `FREQ = 16`.

2. **Clock Control Register (`CCR`):**

- **Standard Mode (100 kHz, 50% duty cycle):**

$$T_{\text{high}} = T_{\text{low}} = \text{CCR} \times T_{\text{PCLK1}} \implies \text{CCR} = \frac{f_{\text{PCLK1}}}{2 \times f_{\text{SCL}}}$$

For $f_{\text{PCLK1}} = 16\text{ MHz}$ and $f_{\text{SCL}} = 100\text{ kHz}$: $\text{CCR} = 16{,}000{,}000 / 200{,}000 = 80$ (`0x0050`).

- **Fast Mode (400 kHz, Duty = 0, $T_{\text{low}} = 2 \times T_{\text{high}}$):**

$$\text{CCR} = \frac{f_{\text{PCLK1}}}{3 \times f_{\text{SCL}}}$$

3. **Rise Time Register (`TRISE`):**

According to I2C specifications, maximum allowable SCL rise time is 1000 ns in Standard Mode and 300 ns in Fast Mode:

$$\text{TRISE}_{\text{SM}} = \frac{1000\text{ ns}}{T_{\text{PCLK1}}} + 1 = \left(\frac{f_{\text{PCLK1}}}{1{,}000{,}000}\right) + 1 = 16 + 1 = 17$$

4.3 Master Transmit & Receive Sequences

```
I2C Master Transmit Sequence:
[START] -> Wait SB=1 -> Write [SlaveAddr + W] -> Wait ADDR=1 -> Clear ADDR (Read SR1, SR2)
        -> For each Byte: Wait TXE=1 -> Write DR -> Wait BTF=1 -> [STOP]

I2C Master Receive Sequence (Len > 1):
[START] -> Wait SB=1 -> Write [SlaveAddr + R] -> Wait ADDR=1 -> Clear ADDR (Read SR1, SR2)
        -> Read Bytes: Wait RXNE=1 -> Read DR
        -> At (Len == 2): Disable ACK -> Generate STOP -> Read Byte 1 -> Wait RXNE -> Read Byte 2
```

- **Repeated Start Support (`Sr`):** If `Sr == I2C_ENABLE_SR`, the driver avoids generating a STOP condition at the end of the transaction. The master maintains exclusive control of the bus to immediately execute an address phase read (essential for reading I2C sensor registers like LM75).
- **Clearing the ADDR Flag:** Handled by a strict hardware sequence: reading `SR1` followed by reading `SR2`. The driver reads both into a volatile dummy variable:

`c

dummy_read = pI2CHandle->pI2Cx->SR1;

dummy_read = pI2CHandle->pI2Cx->SR2;

(void)dummy_read;

`

---

5. SPI Driver Deep Dive

5.1 Hardware Register Architecture

The Serial Peripheral Interface (SPI) is a synchronous, high-speed, full-duplex 4-wire serial bus:

- `CR1`: Bidirectional mode (`BIDIMODE`), Receive-only (`RXONLY`), Software slave management (`SSM`), Internal slave select (`SSI`), Data frame format (`DFF`: 8/16-bit), Baud rate prescaler (`BR[2:0]`: DIV2 to DIV256), Master selection (`MSTR`), Clock polarity (`CPOL`), Clock phase (`CPHA`).
- `CR2`: SSOE (SS Output Enable), TXEIE, RXNEIE, ERRIE.
- `SR`: `BSY` (Busy), `OVR` (Overrun), `MODF` (Mode fault), `TXE`, `RXNE`.
- `DR`: 16-bit data register shared between TX and RX shift registers.

5.2 Slave Select Management (SSM vs Hardware NSS)

In multi-master systems, if the hardware NSS pin is pulled low externally while the SPI is configured as master, the hardware flags a **Mode Fault (`MODF`)** error and forces the peripheral into slave mode (`MSTR` bit cleared in CR1).

To prevent this in single-master configurations without dedicating hardware NSS pins:

1. The driver sets `SPI_SSM = SPI_SSM_EN` (Software Slave Management enabled).

2. The driver sets `SPI_SSIConfig(pSPIx, ENABLE)`: This pulls the internal NSS signal high via the `SSI` bit in `CR1`, guaranteeing the hardware never enters a Mode Fault condition.

3. The physical Chip Select line is controlled manually via a GPIO pin (e.g., `PB12`), giving the firmware precise framing control.

5.3 8-bit vs 16-bit Data Transfer & Pointer Handling

When `DFF = 0` (8-bit frames), data is loaded into `DR` as single bytes.

When `DFF = 1` (16-bit frames), data is cast to `uint16_t*` and written in a single 16-bit half-word write.

(Driver Correction): During our deep-dive analysis, lines 196 & 237 were corrected from invalid `(uint16_t*)pTxBuffer++;` casting to proper multi-byte pointer increments:

```
if (pSPIx->CR1 & (1 << SPI_CR1_DFF)) {
    pSPIx->DR = *((uint16_t*)pTxBuffer);
    Len -= 2;
    pTxBuffer += 2; /* Correct 16-bit address advancement */
} else {
    pSPIx->DR = *pTxBuffer;
    Len--;
    pTxBuffer++;
}
```

5.4 Peripheral Shutdown Protocol

To prevent truncating the final clock pulse or data bit on MOSI:

```
/* 1. Transmit buffer */
SPI_SendData(pSPIx, data, len);
/* 2. Wait until TXE and BSY are cleared */
while (SPI_GetFlagStatus(pSPIx, SPI_BUSY_FLAG));
/* 3. Safely de-assert Chip Select & disable peripheral */
GPIO_WriteToOutputPin(BOARD_SPI_GPIO_PORT, BOARD_SPI_NSS_PIN, GPIO_PIN_SET);
SPI_PeripheralControl(pSPIx, DISABLE);
```

---

6. Board Adaptation & Pin Mapping (Discovery vs Nucleo)

Reference example code originally written for the STM32F4-Discovery board must be ported to the **NUCLEO-F446RE / NUCLEO-F401RE**. The table below highlights physical differences reconciled via `BoardConfig.h`:

Function	STM32F4-Discovery	NUCLEO-F446RE / F401RE	Architectural Impact
User LED	`PD12` (Green, Active HIGH)	`PA5` (LD2 Green, Active HIGH)	Port D on AHB1 vs Port A on AHB1
User Button	`PA0` (Blue, Active HIGH)	`PC13` (B1 Blue, Active LOW)	Active-HIGH pull-down vs Active-LOW pull-up
Debug Serial	Requires external USB-TTL on USART1/2	`PA2` (TX), `PA3` (RX) on USART2	ST-LINK VCP automatically forwards to PC
I2C Bus	I2C1 on `PB6` (SCL), `PB7` (SDA)	I2C1 on `PB8` (SCL), `PB9` (SDA)	Arduino Uno Rev3 Header compatibility
SPI Bus	SPI1 on `PA5`, `PA6`, `PA7`	SPI2 on `PB12`, `PB13`, `PB14`, `PB15`	**Avoids PA5 conflict with User LED!**

Real-World Engineering Gotcha: The PA5 Conflict

On Nucleo boards, `PA5` is physically hardwired to `LD2` LED. If `SPI1` is used with default pins (`PA5 = SPI1_SCK`), toggling SPI clock forces `LD2` to flicker furiously and distorts the clock rise time due to LED capacitance.

Resolution: By mapping `BoardConfig.h` to **SPI2** (`PB12` NSS, `PB13` SCK, `PB14` MISO, `PB15` MOSI), both the User LED and SPI communications operate concurrently without electrical conflict.

---

7. Mandatory Example Applications

All applications are fully implemented, verified, and integrated into the multi-target Makefile:

7.1 GPIO Applications

1. **Button Input with LED Debounce (`examples/example_gpio_button_led.c`):**

- Polls User Button `PC13` (active LOW).

- Executes a 30 ms debouncing delay on edge detection.

- Toggles User LED `PA5` and waits for button release to prevent oscillation.

2. **Software PWM LED Brightness Fade (`examples/example_gpio_pwm_fade.c`):**

- Implements a 500 Hz software PWM loop (`2000 µs` period, 100 duty cycle steps).

- Smoothly ramps duty cycle up from 0% to 100% and back down in a continuous triangular breathing pattern.

7.2 USART Applications

3. **Interactive Character Echo Server (`examples/example_usart_echo.c`):**

- Configures USART2 on `PA2`/`PA3` at 115200 baud, 8N1.

- Transmits startup banner over ST-LINK Virtual COM Port.

- Continuously receives characters and echoes them back with carriage-return line-feed processing.

4. **Structured Telemetry Frames (`examples/example_usart_structured_msg.c`):**

- Transmits periodic 1 Hz heartbeat telemetry frames:

```
[HEARTBEAT] Uptime = 14 s | Status: NORMAL | Total Presses = 3
```

- Detects button presses and instantly transmits asynchronous notification packets:

```
[EVENT] User Button PRESSED! Count = 4 | LED = ON
```

7.3 I2C Applications

5. **LM75 Temperature Reader (`examples/example_i2c_lm75_uart.c`):**

- Queries LM75 temperature sensor at 7-bit address `0x48` over I2C1 (`PB8`/`PB9`).

- Writes register pointer `0x00` with repeated start (`I2C_ENABLE_SR`), then reads 2-byte signed temperature.

- Formats and prints temperature telemetry over USART2:

`[12] [LM75 HARDWARE] Temp: 24.50 C (MSB: 0x18, LSB: 0x80)`

- Includes graceful bench simulation fallback if no external hardware is plugged into the bus.

6. **I2C 7-Bit Bus Address Scanner (`examples/example_i2c_scanner.c`):**

- Probes addresses `0x01` through `0x7F`.

- Sends start condition and address write byte, monitoring `ADDR` (ACK) and `AF` (NACK) flags.

- Prints a formatted ASCII memory grid of all detected slave devices over USART2.

7.4 SPI Applications

7. **Logic Analyzer Test Pattern Generator (`examples/example_spi_test_pattern.c`):**

- Transmits deterministic test bursts on SPI2 (`PB12` CS, `PB13` SCK, `PB15` MOSI).

- Generates alternating preamble patterns (`0xAA, 0x55`), counting ramps (`0x00` to `0xFF`), and magic packets (`0xDEADBEEF`).

- Delimits frames with manual active-LOW chip select for clean triggering on logic analyzers.

8. **Full-Duplex SPI Loopback Verification (`examples/example_spi_loopback_verify.c`):**

- Connects MOSI (`PB15`) directly to MISO (`PB14`) with a jumper wire.

- Transmits test strings while simultaneously reading back bytes from `DR`.

- Verifies byte-by-byte data integrity and outputs Pass/Fail statistics to USART2.

---

8. Personal Reflection & Learning Insights

What Was Easy vs. Challenging

Writing the GPIO driver was straightforward due to the orthogonal nature of the `MODER`, `ODR`, and `IDR` registers. In contrast, the I2C driver presented significant complexity. The STM32 I2C hardware architecture relies heavily on specific register read sequences (such as clearing `ADDR` by reading `SR1` followed by `SR2`) and hardware clock stretching. Overlooking the exact sequence where `ACK` must be cleared *before* reading the final byte causes the master to improperly ACK unwanted bytes on the bus.

Board Porting Experience

Porting code between the STM32F4-Discovery and Nucleo boards highlighted the critical role of a clean board abstraction header (`BoardConfig.h`). The active-LOW button polarity on Nucleo (`PC13` pulled to GND) required inverting the logic threshold used for Discovery's active-HIGH button (`PA0`). Furthermore, the hardware pin conflict

on `PA5` demonstrated that peripheral multiplexing must always be verified against board schematics.

Core Takeaways on Embedded Driver Design

Bare-metal driver development provides an unparalleled appreciation for:

1. **Clock Tree Discipline:** Peripherals cannot function without their corresponding RCC clock gate being explicitly enabled.

2. **Defensive Bit-Masking:** Read-modify-write operations must always clear the target bit fields before writing new values to avoid corrupting adjacent register fields.

3. **Hardware Abstraction Boundaries:** Clean separation between peripheral driver logic (`stm32f446xx_*.c`) and application board wiring (`BoardConfig.h`) enables seamless portability across microcontrollers.